Currency Intelligence 作業報告 2 — IMPLEMENTATION SPEC 02 対応

対象: 2026-10-28 07:10 〜 08:50 UTC / ブランチ: `akihair61/ciel-site` の `currency-intelligence`(main・Pages・ドメインには触れていない)/ コミット `a338f00` → `0f455a1`(push 済み)/ アーティファクト第 12 版

6 / 6	36 / 36	48 / 48	0
Q-01〜Q-06 修正・試験済み	自動テスト(既存 29 の期待値は不変)	ブラウザ自動確認(Playwright、Phase 1 + 2)	本番有効な機能(配信先未決)

報告の約束(SPEC 02 §報告ルール):「作成」= 書いた、「ビルド」= この環境で tsc / esbuild が通った、「試験」= `node --test` か Playwright で確認した、「本番有効」= 利用者が実際に使える配信。**本番有効なものは現時点でひとつもない。**Claude アーティファクトは外部 API に届かないデモ(同梱サンプルレート)であり、独立リポジトリと GitHub Pages はオーナーの操作待ち。

1. 何をしたか(SPEC 02 の順序どおり)

フェーズ	内容	作成	ビルド	試験	本番有効
Phase 0 現物監査	`docs/BASELINE.md`: 監査時 HEAD `9459285`、Node v22.22.0、tsc 6.0.2、29/29 テスト、依存ゼロ、外部 API 到達不可を実測記録。Q-01〜Q-06 の所在(ファイル・行)を特定	✅	—	—	—
Phase 0 分離計画	`docs/MIGRATION_PLAN.md`: private リポジトリ既定、subtree split、Pages は base path 確認後にオーナーが有効化、オリジン変更時は Export/Import、v3→v4 のスキーマ移行表。**実行は未**(このセッションからのリポジトリ作成は GitHub API 403)	✅	—	—	未実行
Phase 0 ADR	`docs/adr/0003-spec02-conflicts.md`: Reduce Motion、「Live」表現、シミュレーション vs 実記録、REDUCED フラグ、数値精度、同梱値の呼称の 6 点について、何を捨て何を残したかを記録	✅	—	—	—
Phase 1 Q-01	数値を double に通さない JSON パーサ `packages/core/src/json.ts`(サイズ 2MB・深さ 64・`__proto__` 無害化)。全プロバイダが `res.text()` → パーサ。24 桁のレートが欠損なく通ることをテスト	✅	✅	✅ 4+1 件	—

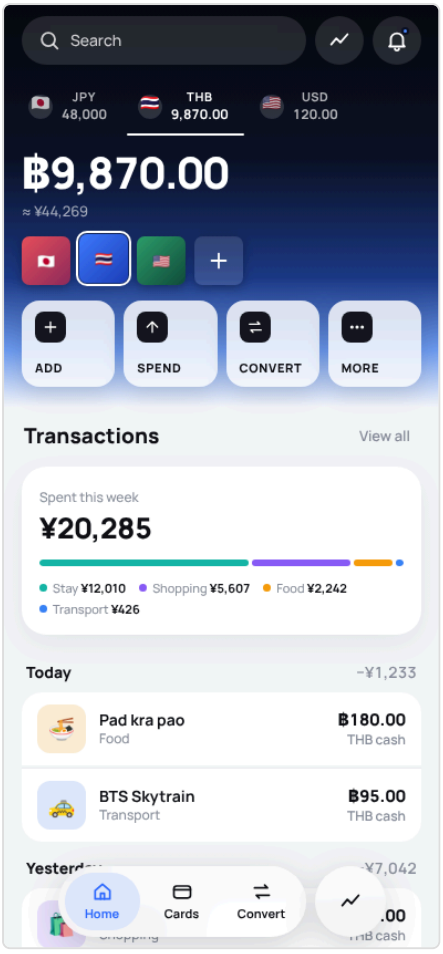
Phase 1 Q-02	`RateQuote.observedAt` を `string | null` に。Coinbase は null、`staleness()` は `basis` を返し、`cross()` は片脚不明なら estimated。UI:「Coinbase · fetched Ns ago · observation time not published」「ECB reference · daily」「Sample rate · not current」。「Live」の語を全廃、レート詳細に Freshness 行	✅	✅	✅	—
Phase 1 Q-03	OS の Reduce Motion を既定で尊重(`motion: auto`)。設定シートで On / Off。reduced 時は全アニメーション 1ms、スプラッシュ・変換ローディング・印字の待ちをスキップ。Playwright の `reducedMotion: "reduce"` でホーム表示 107ms、結果シート 283ms を確認	✅	✅	✅	—
Phase 1 Q-04	Convert = **シミュレーション**(残高・台帳に触れない、参照 `SIM-`)。結果シートに「Simulation · no balance changed, no money moved」タグと「Record as a real exchange」。確認ダイアログ後、2 脚と残高を 1 回の保存で書く(`EXC-`)。既存の変換は `legacy_convert` / `legacy:true` として保持し「Made before v4」と表示。ホーム・カードに「手動で記録した現金であり金融口座残高ではない」。レシートは SIMULATION NOTE / EXCHANGE RECORD / LEGACY CONVERSION	✅	✅	✅	—
Phase 1 Q-05	`settle()` の説明とレシート/割り勘 UI を「精算回数を減らす提案(最少とは限らない)」に。不変条件テスト追加(転送数 ≤ 債務者+債権者−1、収支ゼロ、決定的)	✅	✅	✅	—
Phase 1 Q-06	localStorage `ci.v4`(`schemaVersion: 4`)。保存前に直前値を `ci.v4.bak` に退避。v3 → v4 移行(`ci.v3` は削除しない)、読み取り不能時はバックアップから復旧。Export JSON(封筒 + 版数)/ Import JSON(スキーマ検証・件数提示・確認後に置換)。アーティファクト内では viewer の保存確認を使う	✅	✅	✅	—
Phase 2A F13	ホーム MORE →「Split a bill」: 人数(最大 20)、均等 / 重み、支払者、各人の分担(丸め差の割当表示)、精算提案、共有テキスト、支払者の支出として記録へ連携。残高・台帳は変更しない	✅	✅	✅ **ブラウザ**	—
Phase 2B F11	旅行(名前・予算[ホーム通貨]・期間、選択/終了/削除)。支出・現金追加は **記録時のレートスナップショット**(ホーム通貨額・rate・source・observedAt/receivedAt)と tripId を保持。ホームに旅行カード(支出/予算/残り)。換算基準 = 記録時レート(既定)または現在レート	✅	✅	✅ **7 項目**	—
Phase 2C F01	通貨ピッカーの ★ でピン留め(最大 12、先頭に表示)。UI 言語 **en / ja / th**(`i18n.js` 約 230 語句。端末言語に追従、設定で固定)。未訳: 印字ラベル、ブランド名、出典行の一部、通貨名の th	✅	✅	✅ **8 項目**	—

Phase 2C F03 / F02	All rates シート: 入力金額をピン留め通貨へ同時換算、各行に出典・鮮度、タップで換算先に設定。レート詳細に変動率・日次点・休日・1D の定義文	✅	✅	✅ 3 項目	—
Phase 3 iOS	`ios/CoreMoney` (43 通貨、observedAt null 可、invert/cross、Coinbase、Web と同じ schema v4 の状態ストア)と SwiftUI アプリ `ios/CurrencyIntelligence` (Home / Cards / Convert = シミュレーション / 実両替記録 / レシート / レート詳細 / 設定・Export/Import)。**GitHub Actions の macOS ランナー**(macos-15, Xcode 16.4)で `swift test` 11/11、`xcodebuild` BUILD SUCCEEDED、iPhone シミュレータで起動してスクリーンショット取得。未: XCUITest、実機、TestFlight	✅	✅	🟡 単体 11 件 + 起動	—

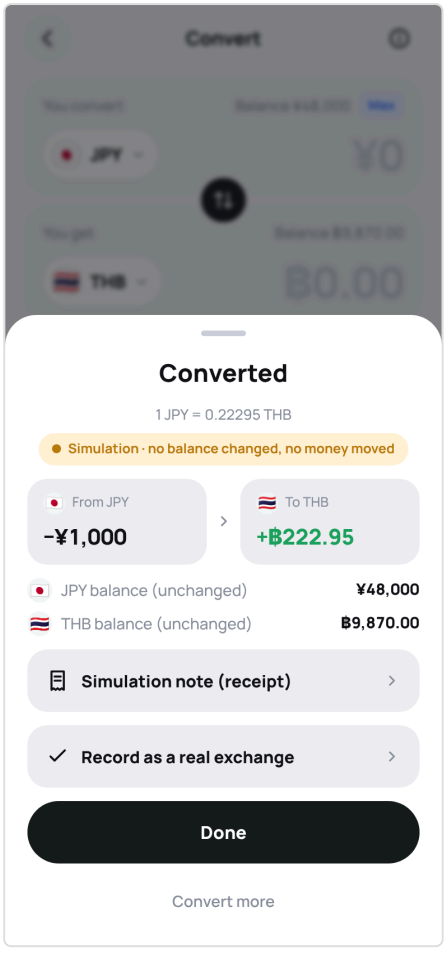
2. 検証の実態(何で確かめ、何は確かめていないか)

確認したこと	方法	結果
計算エンジン	`npm run typecheck`、`node --test test/*.test.ts` (Node 22.22、依存ゼロ)	36 / 36 通過
v3 データの移行	Playwright: `ci.v3` を仕込み再読込 → `ci.v4` 生成、履歴 legacy 化、残高保持、`ci.v3` 無変更	7 項目通過
シミュレーション / 実記録	変換後に残高・台帳が不変、履歴 `SIM-`。Record 後に JPY −500、THB +換算額、台帳 `EXC-`、バックアップに変更前状態	11 項目通過
Export / Import	ダウンロード内容の検証、不正ファイル(残高が数値)拒否、新しい版数拒否、正常ファイルの置換	4 項目通過
Reduce Motion	Playwright `reducedMotion: "reduce"` と設定 On の上書き	3 項目通過
Phase 2(旅行・ピン・All rates・言語)	`docs/verify_p2.js` : 旅行作成 → 支出にスナップショット → 基準切替 → 旅行終了、ピン 3 通貨 → All rates → 行タップ、ja / th / auto の表示と再起動後の保持	20 項目通過
iOS	`.github/workflows/ios.yml` (このブランチへの push で実行)。初回は Import の封筒判定で 1 件失敗 → 修正して 11/11。シミュレータ起動画面は CI がブランチにコミット (`docs/screenshots/ios/ios-home.png`、`docs/ci/ios-build-summary.txt`)	11 / 11、BUILD SUCCEEDED、起動確認
参照フローの録画	`record.js` (起動 → ホーム → カード → 現金追加 → 変換 → レシート)。ページエラーなし	24 フレーム + webm
外部レート API の実疎通	この環境から api.coinbase.com / open.er-api.com / api.frankfurter.app は proxy 403	未確認(テストのフェイク応答のみ)
ダークモード、VoiceOver、実機 iOS、XCUITest	—	未確認

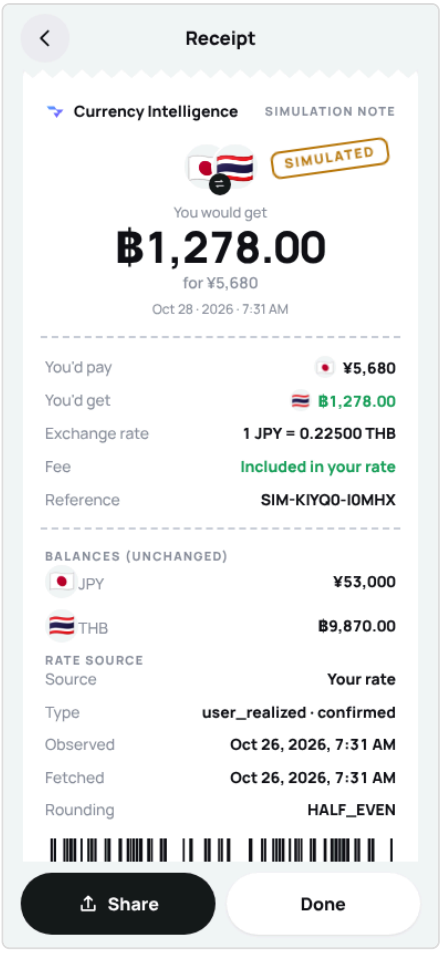
3. 画面(変更後)



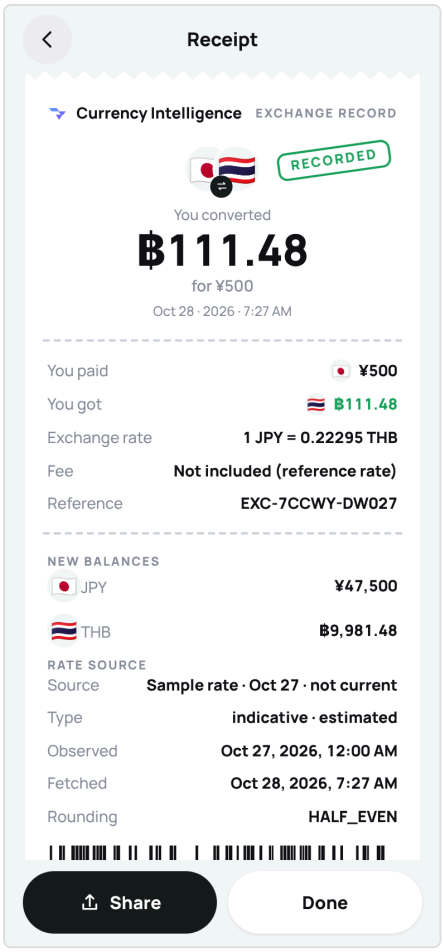
ホーム。「手動で記録した現金」の注記を一覧の下に追加



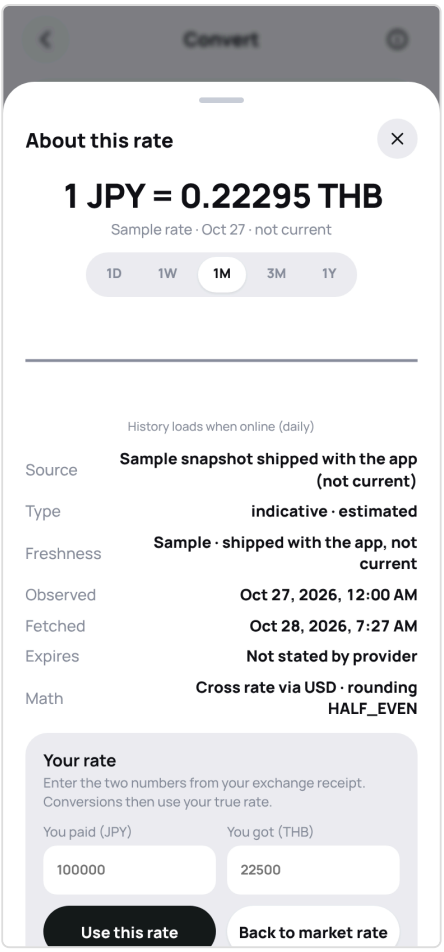
変換結果 = シミュレーション。残高不変のタグと「Record as a real exchange」



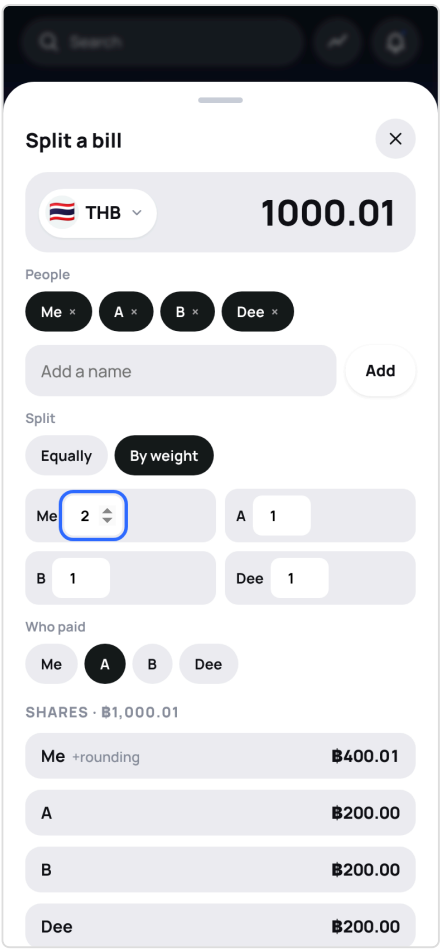
シミュレーション控え(SIMULATED、BALANCES UNCHANGED)



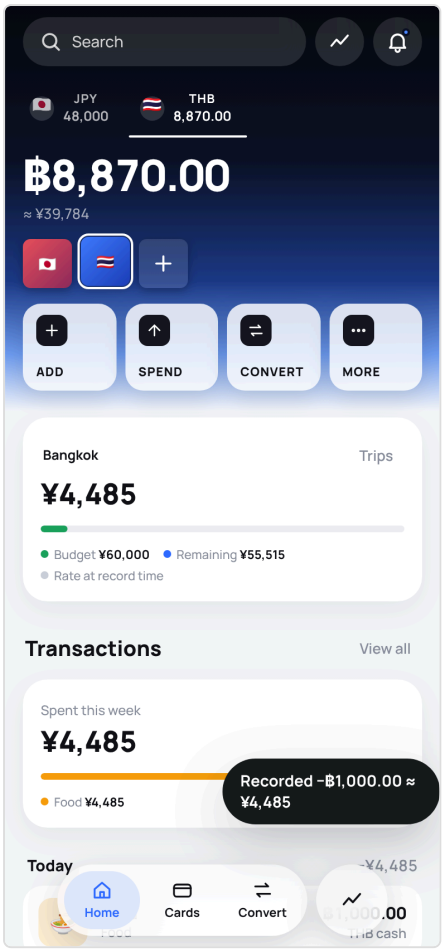
実両替記録(RECORDED、NEW BALANCES、EXC- 参照)



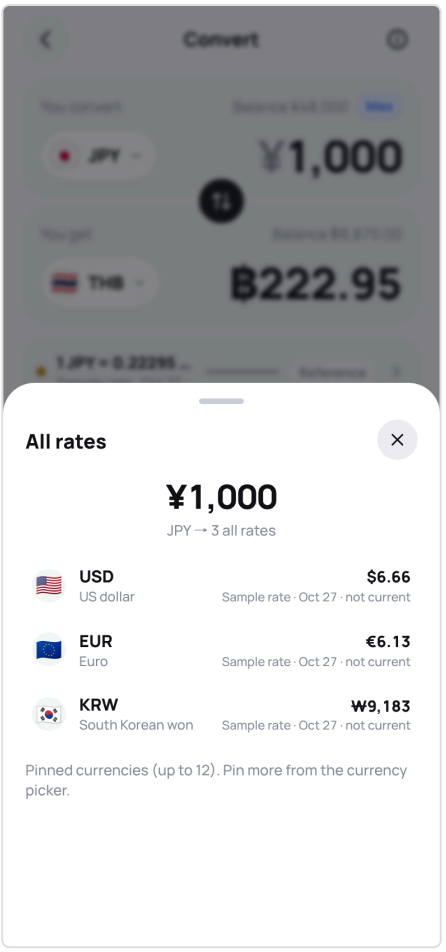
レート詳細: Sample 表示、Freshness 行、観測/取得時刻



割り勘(F13): 重み付き、丸め差の割当、精算提案



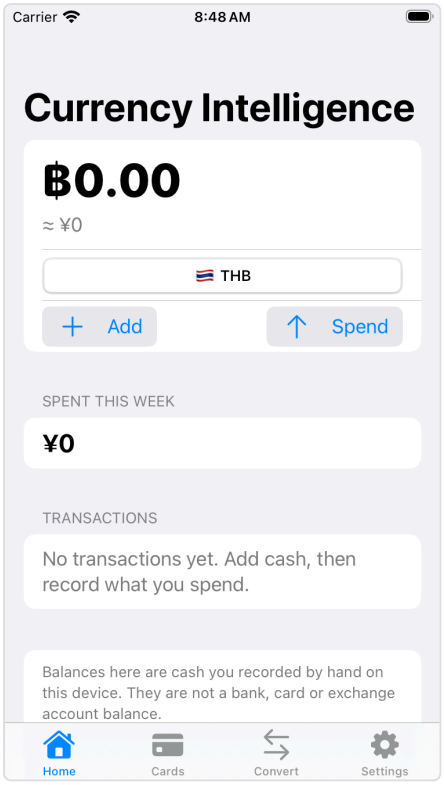
旅行カード(F11): 支出 / 予算 / 残り、記録時レート基準



All rates(F03): ピン留め通貨へ同時換算、各行に出典



日本語 UI(F01)。タイ語も同様



iOS(SwiftUI)。GitHub の macOS ランナー上の iPhone シミュレータで起動した画面

4. 変更したファイル

ファイル	変更
`packages/core/src/json.ts` (新規)、`test/json.test.ts` (新規)	数値保持 JSON パーサ + 4 テスト
`packages/core/src/providers.ts`	`FetchLike` が `text()`、`readJson()`、Coinbase `observedAt: null`
`packages/core/src/rates.ts`, `convert.ts`, `split.ts`, `index.ts`	observedAt null 対応(`staleness.basis`, `quoteTimeMs`)、`receivedAt` を結果に追加、settle の説明、json エクスポート
`packages/core/test/*.test.ts`	フェイク fetch を `text()` に。精度・null・settle 不変条件の 3 テスト追加。既存の期待値は変更なし
`apps/web/app.js`, `index.html`, `styles.css`, `sw.js`	ストレージ v4、motion、文言、シミュレーション/記録、More メニュー、Split シート、設定(Motion / Export / Import)、SW v5
`docs/BASELINE.md`, `MIGRATION_PLAN.md`, `adr/0003`, `FEATURE_MATRIX.md`, `BLOCKERS.md`, `README.md`, `DEPLOY.md`, `TECH_VALIDATION.md`	Phase 0 文書、作成/ビルド/試験/本番有効の列、SPEC 02 対応表、private 既定
`docs/screenshots/spec02/`, `docs/verify_p1.js`, `docs/record.js`	回帰基準の画面・動画と、再現用スクリプト

5. 残っている判断(オーナー)

- **独立リポジトリ**:「全部許可」を受けて作成を再試行したが、GitHub App の権限で拒否された(`POST /user/repos` → 403 Resource not accessible by integration)。GitHub でご自身が private の `currency-intelligence` を作成し、https://claude.ai/connect-github で Claude GitHub App にそのリポジトリを許可してから新しいセッションを始めてください。以後の手順は `docs/MIGRATION_PLAN.md`。
- **配信先**: Pages(private で使えるかはプラン次第)か別ホスティングか。`ciel-site` のドメイン配下は指示がない限り使わない。
- **外部 API の許可**: この環境のネットワーク許可に `api.coinbase.com`, `open.er-api.com`, `api.frankfurter.app` を加えると、プロバイダを実応答で検証できる。
- **見た目の変更の可否**: 結果シートのタグ+ボタン追加、レシートの見出し語は参照動画と異なる(ADR-0003)。

やっていないこと(正直に): iOS の画面操作の自動試験(XCUITest)と実機・TestFlight、リポジトリ分離の実行(GitHub App の権限で 403)、外部 API の実疎通、ダークモードの目視、多言語の未訳部分。これらを「済み」とは書いていない。